Time-LLM 기반 Retrieval-Conditioned Prompt-as-Prefix 강화를 활용한 클라우드 리소스 시계열 예측

2026학년도 1학기 졸업프로젝트

목차

1. 개요

a. 연구 배경

b. 관련 연구

c. 캡스톤디자인 연구 요약 및 한계

d. 문제 정의

2. 제안

a. 연산 병목 해결

b. Dynamic PaP

c. Balanced RAG

3. 실험

- 데이터셋
- 실험 설정
- 실험 설계

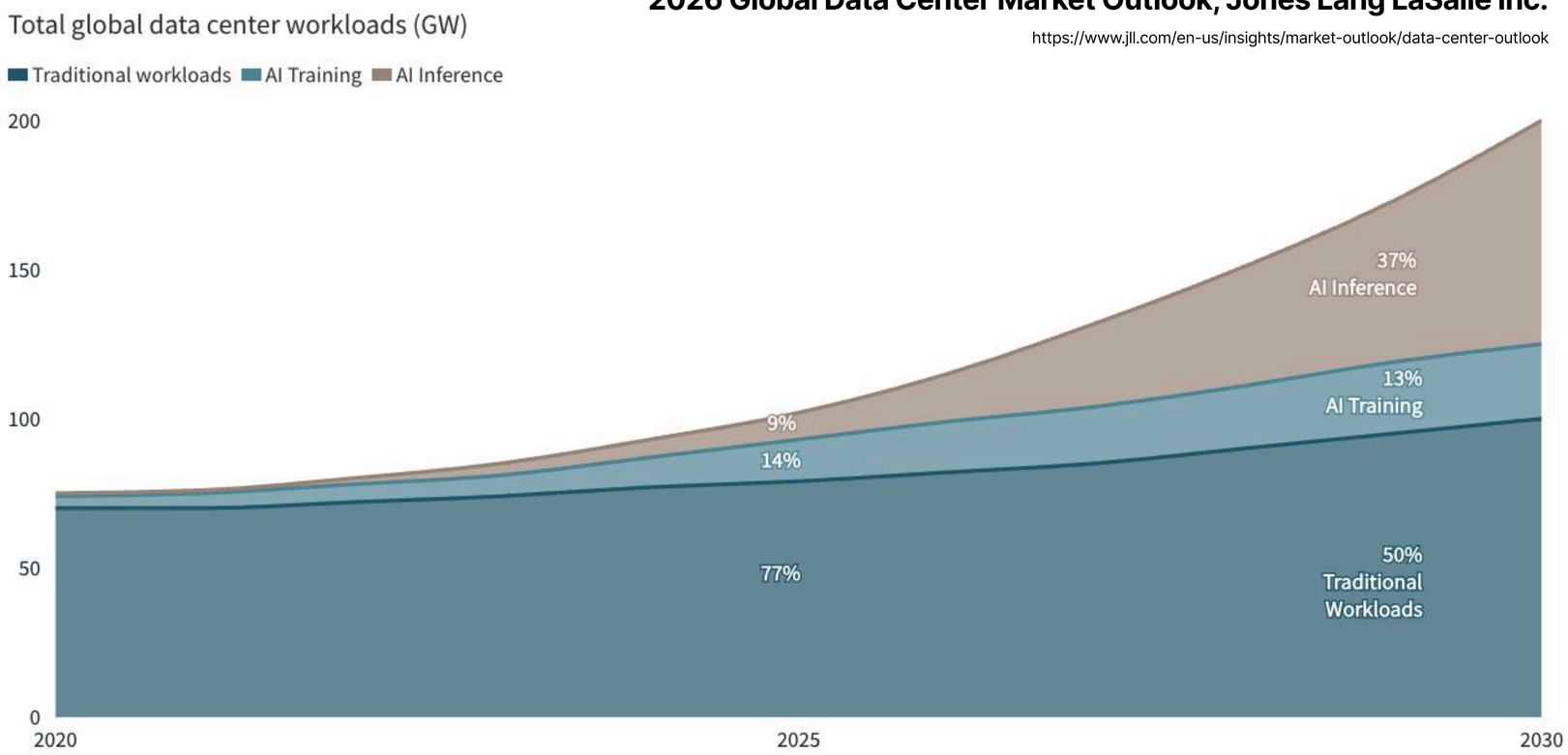
4. 결과

- Ablation Studies
- 분석
- 결론

연구 배경

2026 Global Data Center Market Outlook, Jones Lang LaSalle Inc.

https://www.jll.com/en-us/insights/market-outlook/data-center-outlook



| 데이터센터 사용량은 지속적으로 증가

- 기존 퍼블릭 클라우드 서비스 제공량
- 2030년에는 AI 관련 수요가 50% 이상일 것으로 예상

| DevOps, FinOps

- 클라우드에서 파생된 비즈니스 가치 향상 운영 관리 분야
- Infra와 애플리케이션 복잡성 증가로 생긴 Challenge

연구 배경

Instance-Level SLA

For each individual Amazon EC2 instance ("Single EC2 Instance"), AWS will use commercially reasonable efforts to make the Single EC2 Instance available with an Instance-Level Uptime Percentage of at least 99.5%, in each case during any monthly billing cycle (the "Instance-Level SLA"). In the event any Single EC2 Instance does not meet the Instance-Level SLA, you will be eligible to receive a Service Credit as described below.

Instance-Level Uptime Percentage	Service Credit Percentage
Less than 99.5% but equal to or greater than 99.0%	10%
Less than 99.0% but equal to or greater than 95.0%	30%
Less than 95.0%	100%

Note: In addition to the Instance-Level SLA, AWS will not charge you for any Single EC2 Instance that is Unavailable for more than six minutes of a clockhour. This applies automatically and you do not need to request credit for any such hour with more than six minutes of Unavailability.

Region-Level SLA

For Amazon EC2 with all running instances deployed concurrently across two or more AZs in the same region (or at least two regions if there is only one AZ in a given region), AWS will use commercially reasonable efforts to make Amazon EC2 available for each AWS region with a Monthly Uptime Percentage of at least 99.99%, in each case during any monthly billing cycle (the "Region-Level SLA"). In the event Amazon EC2 does not meet the Region-Level SLA, you will be eligible to receive a Service Credit as described below.

Monthly Uptime Percentage	Service Credit Percentage
Less than 99.99% but equal to or greater than 99.0%	10%
Less than 99.0% but equal to or greater than 95.0%	30%
Less than 95.0%	100%

Amazon Compute Service Level Agreement, AWS

https://aws.amazon.com/compute/sla/

| 클라우드 서비스 제공자의 SLA 준수 필요성 증가

- 월간 서비스 운영률이 *99.99%* 미만일 경우 서비스 크레딧 제공
- IT 산업계의 SLA 미준수 시 페널티는 **$82M**에 육박

The Hidden Costs of Downtime 2026, splunk (cisco company)

| 클라우드 서비스의 자원 관리 필요성

- 자원을 필요 이상 여유있게 제공하는 과다 할당
- 리소스 관리를 위한 노력이 생태계를 만듦
  - Borg, Kubernetes, Nomad

관련 연구

| 시계열 예측에 관한 연구

a. 통계 활용

- **ARIMA, Prophet**
  - 잔차, 추세, 계절성을 바탕으로 예측
  - 빠르며 직관적인 파라미터로 모델 수정이 용이

b. Machine Learning (1)

- **RNN**
  - 이전 시점의 정보가 현재 시점의 처리에 영향을 미치는 순환 구조
- **LSTM**
  - RNN의 장기 기억 단점을 극복하기 위함
  - Memory Cell을 도입하여 수 단계 이전의 정보를 저장

c. Machine Learning (2)

- **PatchTST**
  - 시계열을 패치 단위 토큰으로 관리
- **Time-LLM**
  - 시계열 정보를 텍스트 기반 패치 표현으로 변환

관련 연구

| 클라우드 리소스 관리

a. Cloud Workload Trace data

- Google, Azure, Alibaba,Tencent 등 클라우드 서비스 제공업자들이 오픈소스로 공개한 데이터셋

b. AIOps

- AI 기능을 적용하여 운영을 자동화하고 효율화하며 최적화
- 관측 가능성(Observability)
  - 장애 발생 후 수동 대응에서 선행 대응 및 자동 복구
  - Log, Metric, Event 등의 데이터 수집 및 분석

c. FinOps

- 클라우드 서비스 제공자뿐 아니라 사용자 측에서도 리소스 관리 최적화 필요성이 증가
- 비용 예측 고도화, 실시간 비용 이상 탐지, 자동 최적화에 활용

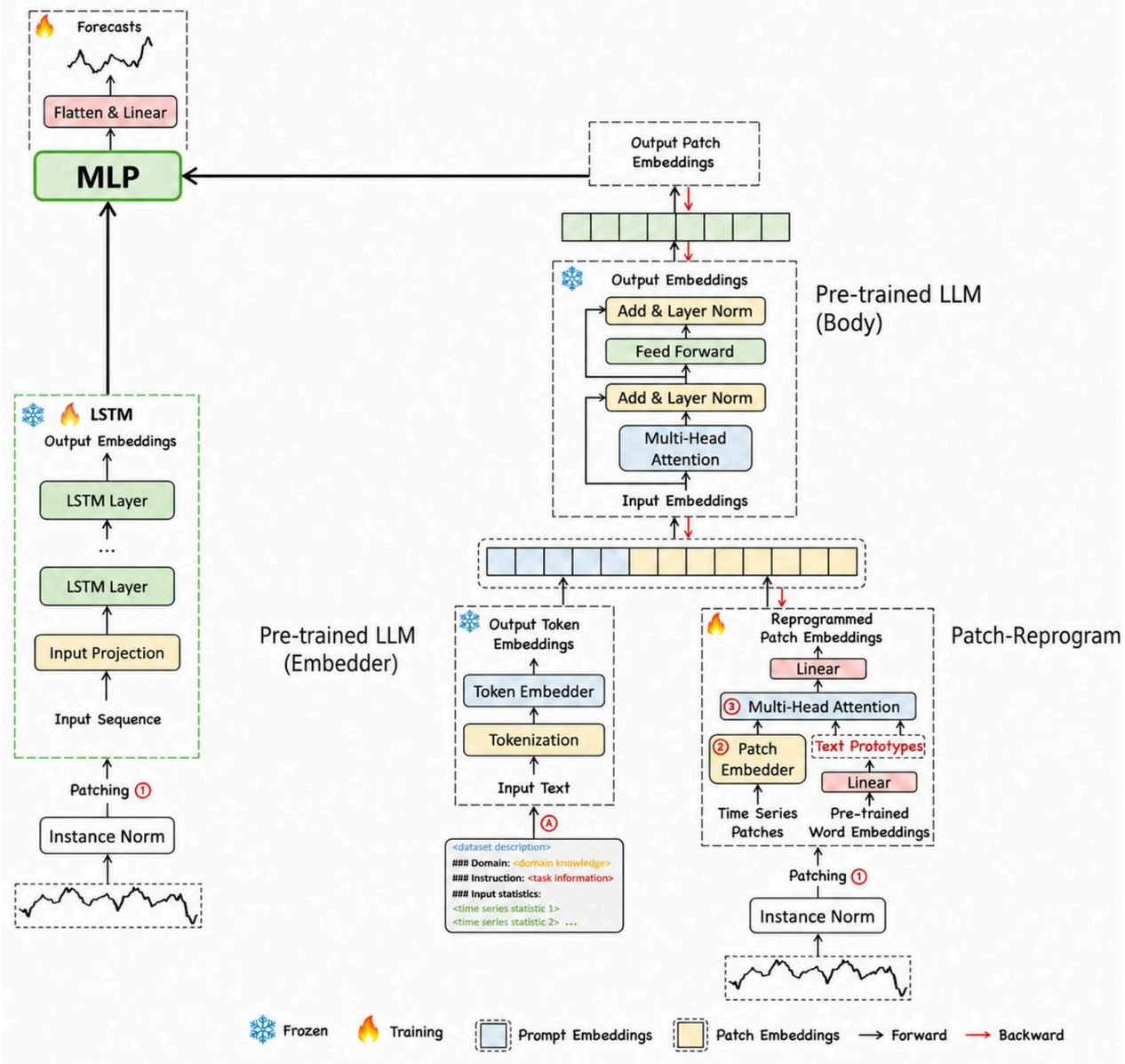
캡스톤디자인 요약

1. Time-LLM + LSTM Hybrid Architecture

- 시계열 패치가 LSTM을 통과하여 출력 생성
- 시계열 패치가 Time-LLM을 통과하여 출력 생성
- 두 경로의 출력이 MLP에서 합쳐져 최종 예측

2. 결과

- Time-LLM의 장기 패턴 학습 능력 활용
- LSTM의 단기 패턴 학습 능력 활용
- **일부 Feature 성능 개선**



캡스톤디자인 요약

Ablation Studies (일부)

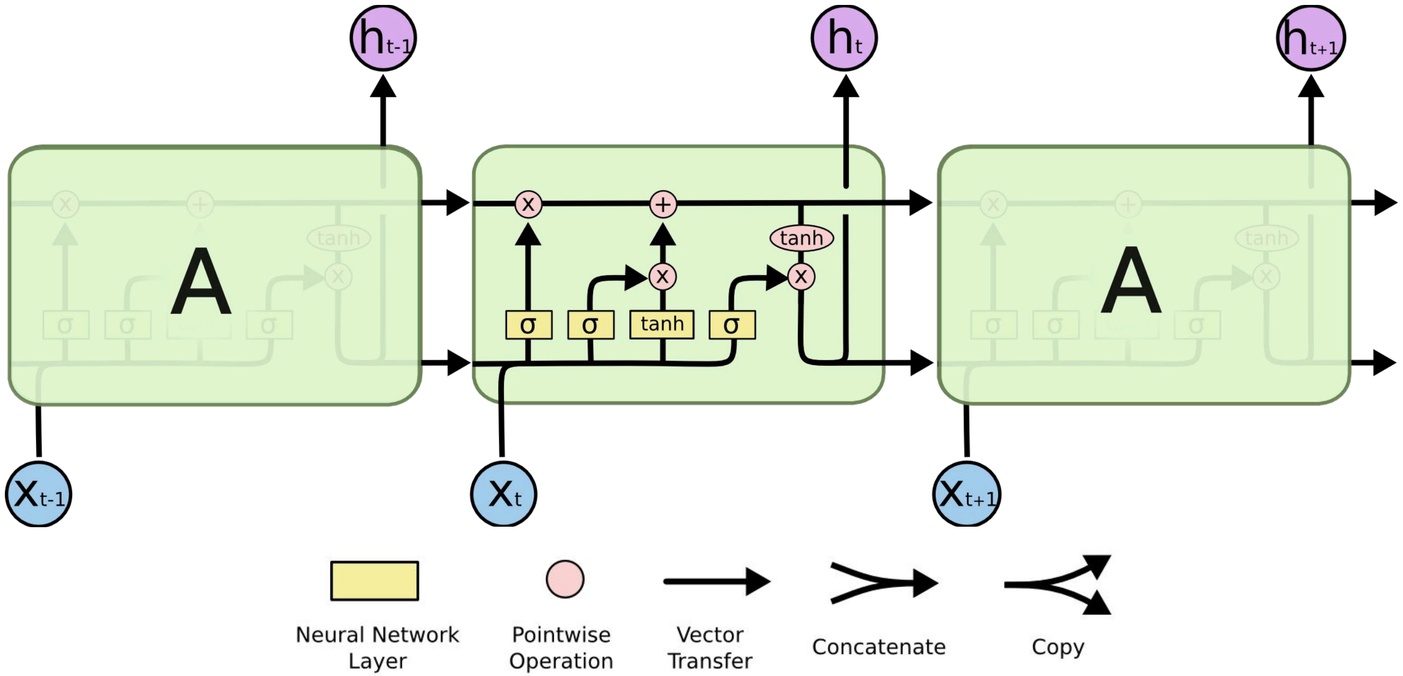
상대 변화율의 음수는 오차 감소, 양수는 오차 증가를 의미

No	Name	MAE	MSE	상대 변화율	MLP Fusion Method
A0	Time-LLM	0.006059	0.000303	0.00% (Base)	
A1	LSTM	0.025163	0.036279	+315.4%	
A2	Bi-LSTM	0.028592	0.038620	+372.2%	
A3	Stacked-LSTM	0.029019	0.038360	+379.1%	
A4	**LSTM + Time-LLM**	**0.003889**	**0.000026**	**-35.8%**	**Concatenation**
A5	Bi-LSTM + Time-LLM	0.014670	0.000238	+142%	Concatenation

캡스톤디자인 한계

1. Time-LLM + LSTM Hybrid Architecture 연산 병목

- LSTM의 본질인 RNN 구조로 인한 병목
- ***t*** 시점의 상태를 계산하려면 ***t-1*** 시점의 연산이 먼저 완료되어야 하는 순차 의존성 발생
- 병렬 환경인 GPU 환경에서 불리



캡스톤디자인 한계

2. Time-LLM의 추론 능력 활용 부족

- Time-LLM의 기여 중 하나인 프롬프트(Prompt-as-Prefix) 활용도가 낮았음
  - 데이터 세트에 대한 개략적 설명만 정적으로 제공
- 윈도우별 통계 문맥 및 도메인 지식을 LLM에 충분히 전달하지 못하였음
- 비정상성 구간에서 LLM 추론 능력 활용도 저하

문제 정의

1. Time-LLM + LSTM Hybrid Architecture의 연산 병목 해결

- LSTM 병렬 경로 제거
- 메모리 및 연산 여유분 확보

2. RAG 기반 동적 프롬프트 주입

- Google Cluster Trace 기반 FAISS 벡터 DB 구축
- 슬라이딩 윈도우마다 유사 과거 패턴과 Future Horizon 검색
- Class-balanced sampling으로 불균형 문제 해소

제안 1 - Hybrid Architecture의 연산 병목 해결

| LSTM 병렬 경로 제거

- Time-LLM 아키텍처를 그대로 활용

| 메모리 및 연산 여유분 확보

- 확보한 메모리를 RAG 도입과 데이터셋에 할당
    - 캡스톤디자인
        - 35,327,512개의 row에서 10%인 **3,532,751 row**의 데이터셋만 학습
    - 졸업프로젝트
        - 소수의 timestamp를 가진 불필요 인스턴스를 제거한 **9,117,645 row**의 데이터셋 전체 학습

제안 2 - Dynamic Prompt-as-Prefix

| 기존 프롬프트 구조

a. 데이터셋 컨텍스트

- *We usually observe that electricity consumption peaks at noon, with a significant increase in transformer load*

b. 작업 지침

- *Predict the next <H> steps given the previous <T> steps information attached*

c. 통계적 컨텍스트

- *The input has a minimum of <min_val>, a maximum of <max_val>, and a median of <median_val>. The overall trend is <upward or downward>. The top five lags are <lag_val>.*

Time-LLM 원 논문 예시

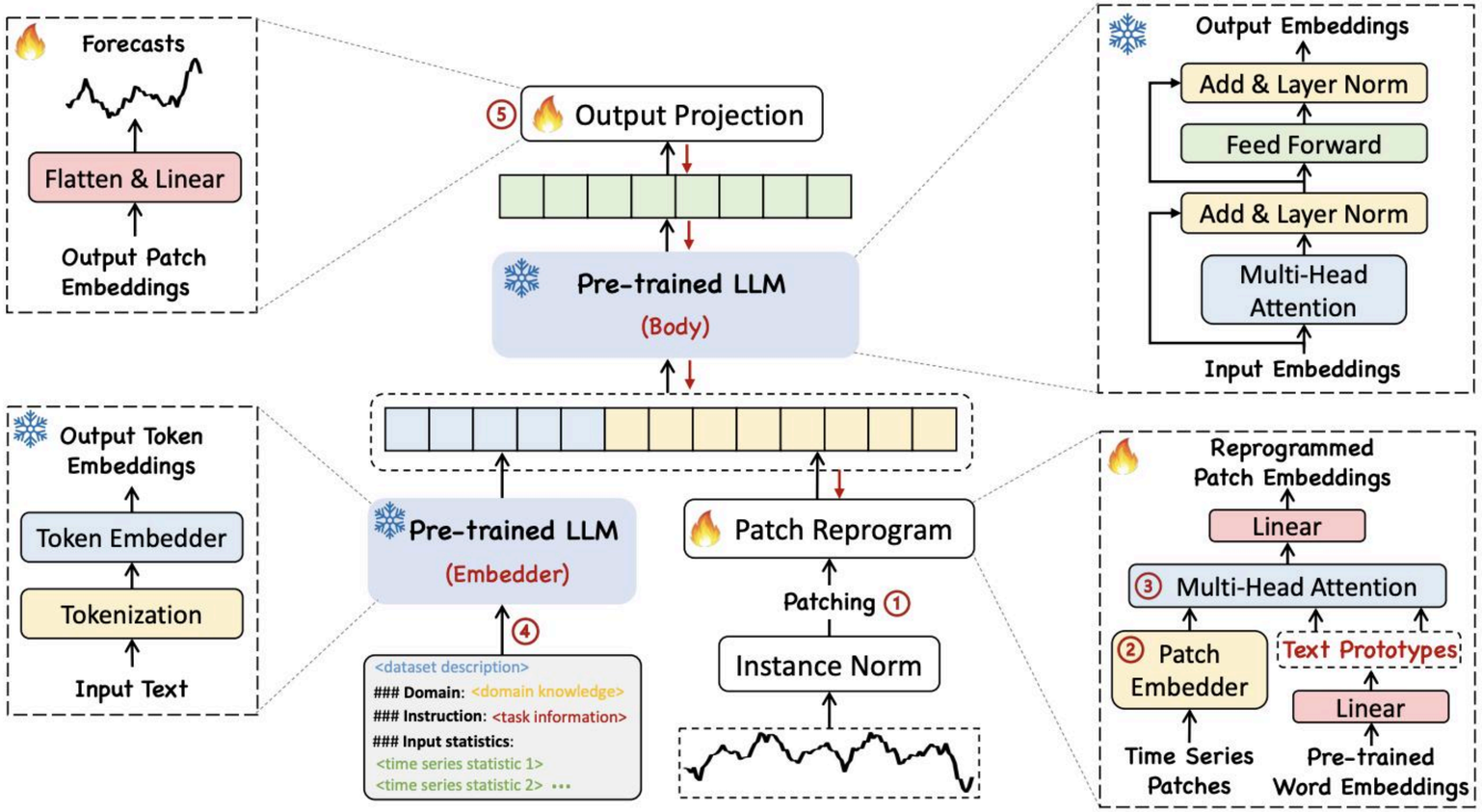
제안 2 - Dynamic Prompt-as-Prefix

| 기존 프롬프트 구조

a. 데이터셋 컨텍스트

b. 작업 지침

c. 통계적 컨텍스트



제안 2 - Dynamic Prompt-as-Prefix

| 제안 프롬프트 구조

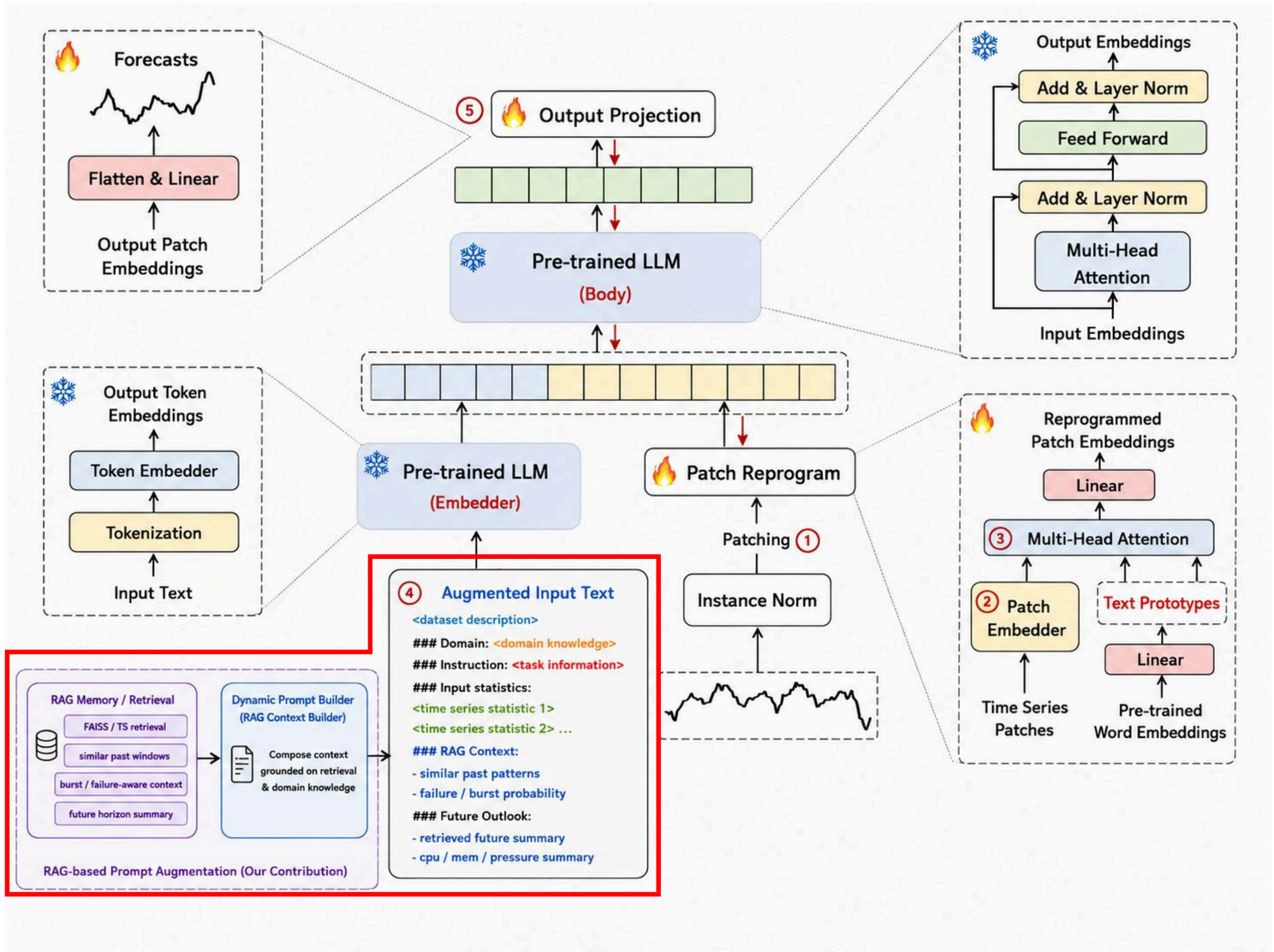
a. 데이터셋 컨텍스트

b. 작업 지침

c. 통계적 컨텍스트

d. **RAG 컨텍스트**

- **Historical context**
- **Future outlook**



제안 2 - Dynamic Prompt-as-Prefix

| 제안 프롬프트 구조

a. 데이터셋 컨텍스트

b. 작업 지침

c. 통계적 컨텍스트

d. RAG 컨텍스트

- **Historical context**
  - *pattern type is burst, failure probability 60% (3 of 5 similar patterns),*
- **Future outlook**
  - *cpu_usage: mean 0.031 max 0.089,*
  - *mem_usage: mean 0.041 max 0.112,*
  - *cpu_pressure: mean 0.412 max 0.891,*
  - *(...)*

제안 2 - Dynamic Prompt-as-Prefix

| RAG DB 생성 프로세스

1. **전체 데이터셋 중 20% 분리**
    a. Data Leakage 방지
    b. RAG DB에 사용된 데이터는 Train, Val, Test에 사용되지 않음
2. **인스턴스 단위로 분리**
3. **클래스 라벨링**
    a. failure-sudden (폭증 후 장애)
    b. failure-gradual (점진적 상승 후 장애)
    c. burst (일시적 상승 후 정상)
    d. normal-rising (상승 추세)
    e. normal-stable (안정)
4. **슬라이딩 윈도우 생성**
    a. 시계열 길이 T
5. **통계 임베딩 추출**
6. **L2 정규화**
    a. 정규화된 벡터에서 L2와 코사인 유사도는 동치
7. **FAISS IndexFlatL2 구축**
    - Facebook AI Similarity Search
    - 고차원 벡터의 in-RAM 유사도 검색
8. **Time-LLM 모델의 프롬프트로 주입**

제안 2 - Dynamic Prompt-as-Prefix

| 프롬프트 반영

```
prompt_ = (
    # 1. Dataset Context
    f"<|start_prompt|>Dataset description: {self.description}"

    # 2. Task Instruction
    f"Task description: forecast the next {str(self.pred_len)} steps given the previous {str(self.seq_len)} steps

    # 3. Statistical Context
    "Input statistics: "
    f"min value {min_values_str}, "
    f"max value {max_values_str}, "
    f"median value {median_values_str}, "
    f"top 5 lags are : {lags_values_str}; "

    # 4. RAG Context
    f"Historical context: "
    f"pattern type is {dominant_cls}, "
    f"failure probability {fail_rate:.0%} ({n_fail} of {top_k} similar patterns), "
    f"future outlook — "
    f"cpu_usage: mean {cpu_usage_mean:.3f} max {cpu_usage_max:.3f}, "
    f"mem_usage: mean {mem_usage_mean:.3f} max {mem_usage_max:.3f}, "
    f"cpu_pressure: mean {cpu_p_mean:.3f} max {cpu_p_max:.3f}, "
    f"mem_pressure: mean {mem_p_mean:.3f} max {mem_p_max:.3f}, "
    f"cpu_burst: max {cpu_b_max:.3f}."
    f"<|<end_prompt>|>"
```

제안 2 - Dynamic Prompt-as-Prefix

| 프롬프트 반영

| Time-LLM 원 논문의 프롬프트 예시
Dataset description:
The Electricity Transformer Temperature (ETT) is a crucial indicator in the electric power long-term deployment.

Task description:
forecast the next 32 steps given the previous 32 steps information;

Input statistics:
min value -2.146, max value 3.821, median value 0.0234,
top 5 lags are : [12, 24, 6, 1, 48]

| 제안 방법론의 프롬프트 예시
Dataset description:
The Google Cluster Trace v3 (Cell a) dataset contains multi-dimensional time-series of cloud resource utilization from Google data centers.

Task description:
forecast the next 32 steps given the previous 32 steps information;

Input statistics:
min value 0.0021, max value 0.0734, median value 0.0183,
top 5 lags are : [1, 2, 12, 24, 6]

Historical context:
dominant pattern is burst, failure rate 60% (3 of 5 similar patterns),

Future outlook:
cpu_usage: mean 0.045 max 0.112,
mem_usage: mean 0.038 max 0.097,
(...)

제안 3 - Balanced RAG

1. 데이터셋 불균형

- 클라우드 리소스 데이터 도메인 특성으로 FAIL 인스턴스와 FINISH 인스턴스의 극심한 불균형
  - FAIL: 약 2.1K (~2%)
  - FINISH: 약 35K (~98%)
- FAIL 인스턴스 전수 사용
- 학습 시 FINISH 인스턴스는 1/4 다운샘플링

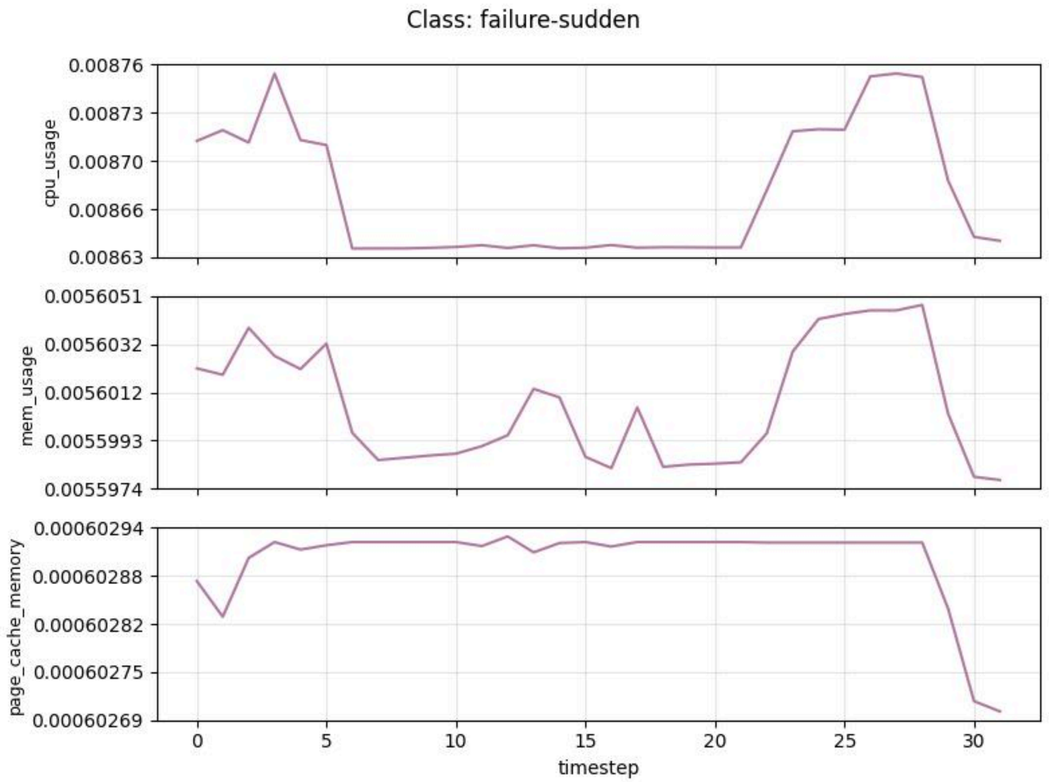
2. 패턴 불균형

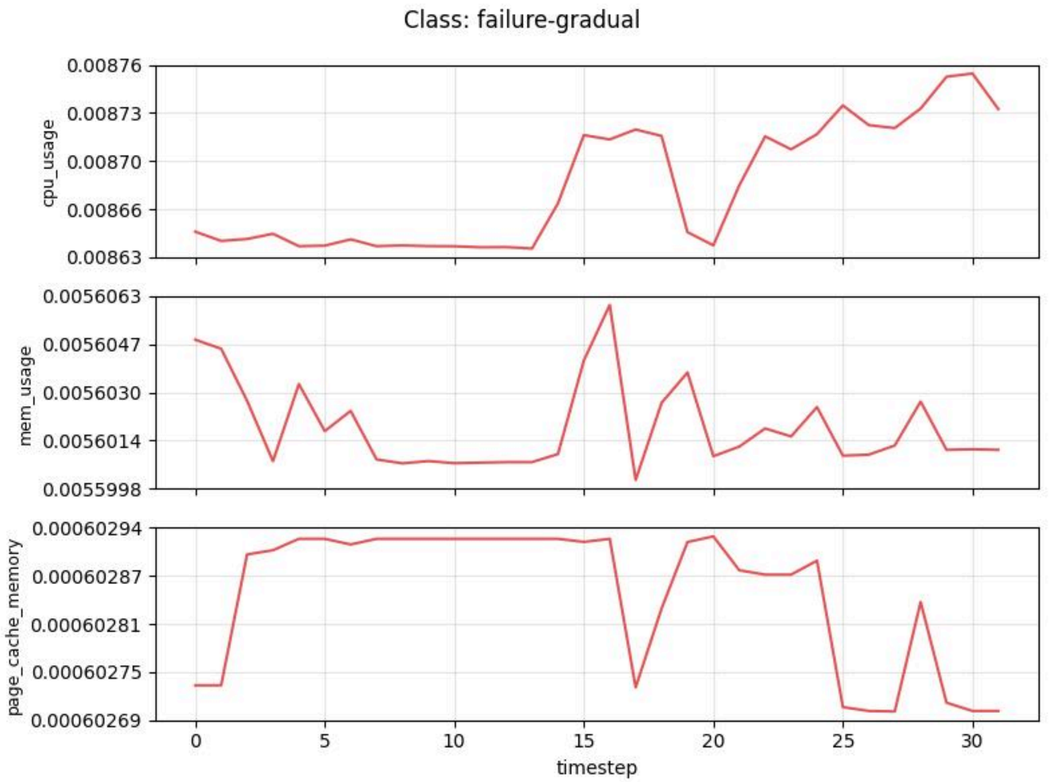
- 클라우드 리소스 데이터 도메인 특성으로 안정적으로 진행하는 패턴이 다수
- 유사도 검색 시 각 클래스에서 top-k 검색이 가능하도록 충분한 윈도우 수 확보

클래스	이름	window 수	비율
0	normal-stable	15,067	31.71%
1	normal-rising	2,588	5.45%
2	**burst**	**23,960**	**50.42%**
3	failure-gradual	3,254	6.85%
4	failure-sudden	2,656	5.59%

제안 3 - Balanced RAG

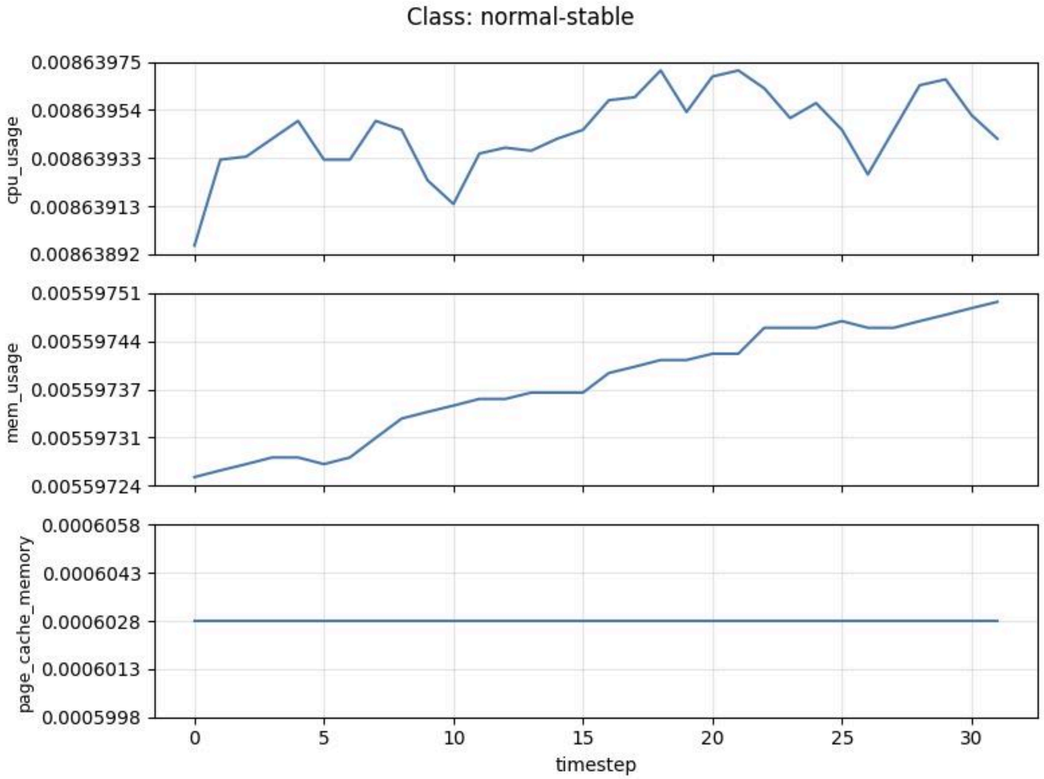
| 클래스별 데이터 양상

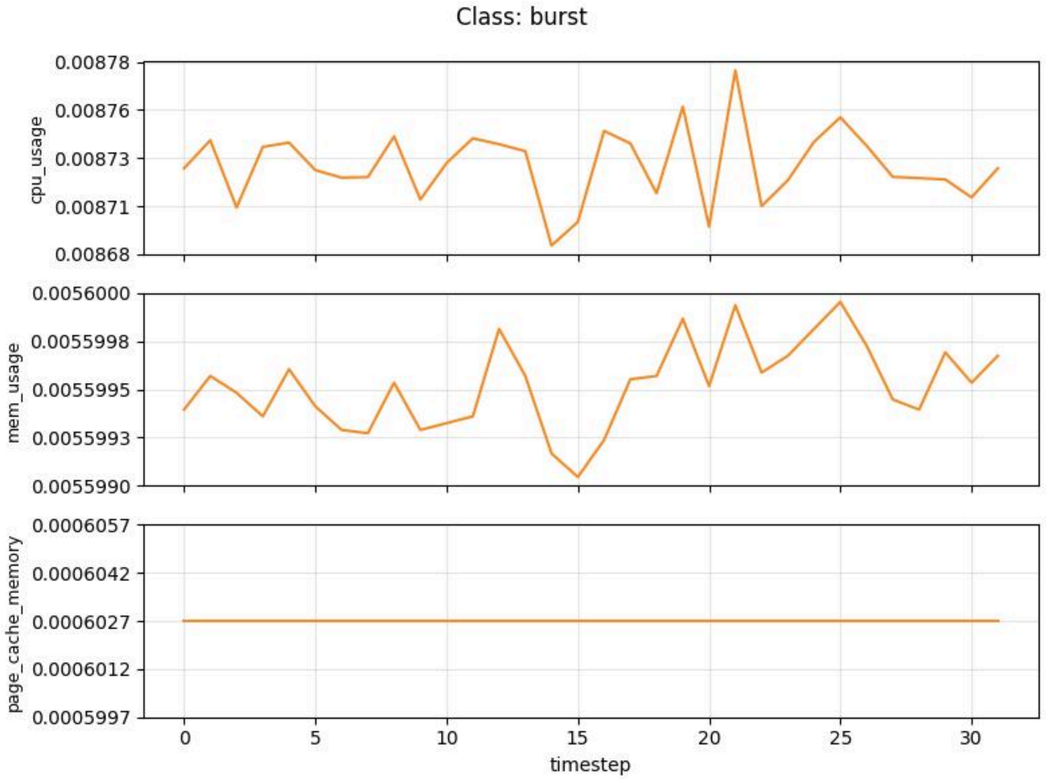




제안 3 - Balanced RAG

| 클래스별 데이터 양상





데이터셋

| ClusterData2019

a. 개요

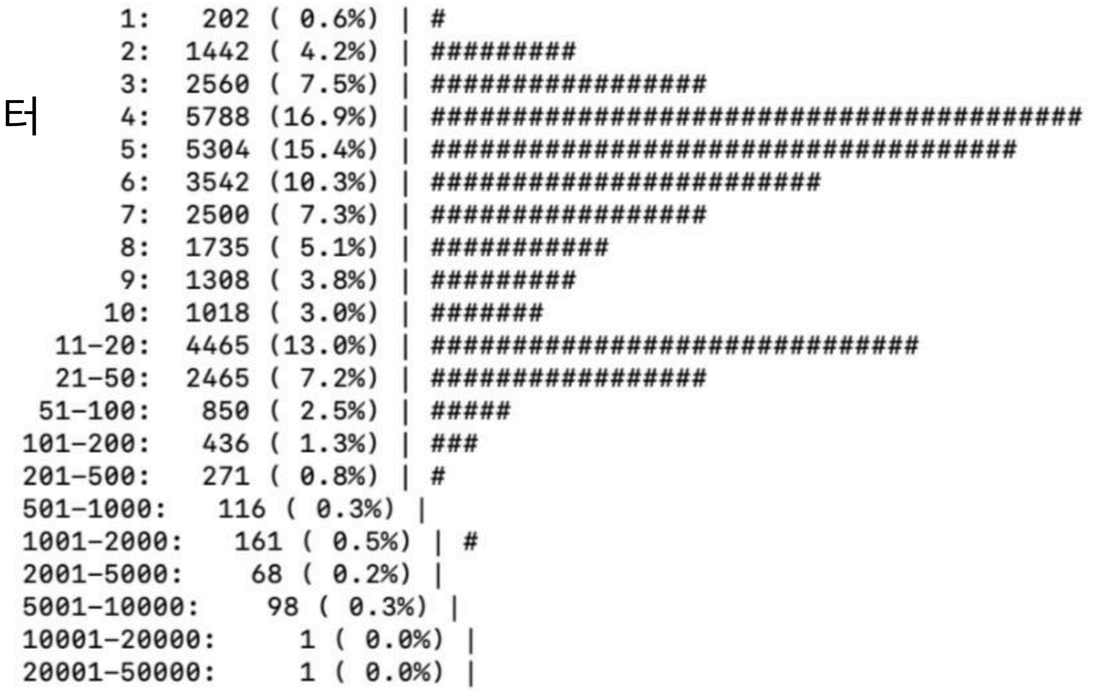
- 2019년 5월 한 달 동안 8개의 데이터센터 지역에서 수집한 데이터

b. 데이터셋 규모

- 8개의 지역 중 a-cell 사용
    - 10,000 machines / 7.4M instances / 7.5B rows

c. 데이터 전처리

- 1시간 미만의 Task가 **81.29%**
- 64개 이상의 row를 가진 인스턴스로 제한



실험 설정

모델 및 학습 설정

- Backbone: GPT-2 (124M params)
- GPU: NVIDIA RTX3090 (single)
  - 24시간 walltime 제약

데이터셋

- Google Cluster Trace v3
  - Cell a
- 10개 feature
  - cpu, mem 사용량 등 포함

비교 방법

- A0 Baseline 측정 후 Ablation 실험 수행

평가 지표

- Prediction - MAE, MSE

실험 설계

No	Name	use_rag	rag_mode	top_k	설명
A0	Time-LLM	X	-	-	Baseline
A1	RAG_top5	O	normal	5	유사 인스턴스 top5 검색 후 프롬프트 주입
A2	RAG_random	O	random	5	top5 랜덤 샘플링
A3	RAG_fixed	O	fixed	5	고정 텍스트 프롬프트 (검색 없음)
A4	RAG_top10	O	normal	10	A1과 동일, top_k=10
A5	RAG_top20	O	normal	20	A1과 동일, top_k=20
A6	RAG_top100	O	normal	100	A1과 동일, top_k=100

Ablation Studies - cpu_usage

cpu_usage

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00005204	—	—	0.00332090	—	—
A1	RAG_PaP	0.00005199	-0.00000005	-0.0997%	0.00330794	-0.00001296	-0.3902%
A2	Random_RAG	0.00005186	-0.00000018	-0.3479%	0.00331290	-0.00000800	-0.2408%
A3	Fixed_Prompt	0.00005197	-0.00000007	-0.1297%	0.00332105	+0.00000015	+0.0045%
A4	RAG_top10*	0.00005199	-0.00000005	-0.0993%	0.00331168	-0.00000922	-0.2778%
A5	RAG_top20*	0.00005201	-0.00000004	-0.0707%	0.00331730	-0.00000360	-0.1083%
A6	RAG_top100*	0.00005209	+0.00000005	+0.1022%	0.00331781	-0.00000309	-0.0930%

Ablation Studies - max_cpu_usage

max_cpu_usage

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00122338	—	—	0.01677282	—	—
A1	RAG_PaP	0.00122468	+0.00000130	+0.1064%	0.01675292	-0.00001990	-0.1186%
A2	Random_RAG	0.00122247	-0.00000092	-0.0749%	0.01684628	+0.00007347	+0.4380%
A3	Fixed_Prompt	0.00122209	-0.00000130	-0.1059%	0.01673032	-0.00004249	-0.2533%
A4	RAG_top10*	0.00122298	-0.00000040	-0.0326%	0.01673560	-0.00003721	-0.2219%
A5	RAG_top20*	0.00122366	+0.00000028	+0.0230%	0.01678174	+0.00000893	+0.0532%
A6	RAG_top100*	0.00121259	-0.00001079	-0.8818%	0.01669012	-0.00008270	-0.4930%

Ablation Studies - mem_usage

mem_usage

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00003475	—	—	0.00167465	—	—
A1	RAG_PaP	0.00003467	-0.00000009	-0.2514%	0.00166351	-0.00001114	-0.6654%
A2	Random_RAG	0.00003464	-0.00000012	-0.3324%	0.00167247	-0.00000218	-0.1305%
A3	Fixed_Prompt	0.00003476	+0.00000000	+0.0112%	0.00168098	+0.00000632	+0.3777%
A4	RAG_top10*	0.00003473	-0.00000003	-0.0733%	0.00167185	-0.00000281	-0.1675%
A5	RAG_top20*	0.00003465	-0.00000010	-0.2863%	0.00167982	+0.00000516	+0.3083%
A6	RAG_top100*	0.00003515	+0.00000039	+1.1346%	0.00166768	-0.00000697	-0.4163%

Ablation Studies - max_mem_usage

max_mem_usage

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00003942	—	—	0.00179866	—	—
A1	RAG_PaP	0.00003937	-0.00000006	-0.1459%	0.00178768	-0.00001097	-0.6102%
A2	Random_RAG	0.00003926	-0.00000016	-0.4012%	0.00179898	+0.00000032	+0.0179%
A3	Fixed_Prompt	0.00003943	+0.00000001	+0.0218%	0.00180480	+0.00000614	+0.3415%
A4	RAG_top10*	0.00003939	-0.00000003	-0.0879%	0.00179597	-0.00000269	-0.1494%
A5	RAG_top20*	0.00003923	-0.00000020	-0.5003%	0.00180109	+0.00000244	+0.1355%
A6	RAG_top100*	0.00003980	+0.00000038	+0.9608%	0.00180525	+0.00000659	+0.3663%

Ablation Studies - page_cache_memory

page_cache_memory

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00000030	—	—	0.00039946	—	—
A1	RAG_PaP	0.00000028	-0.00000002	-7.1863%	0.00039636	-0.00000310	-0.7771%
A2	Random_RAG	0.00000028	-0.00000002	-6.1906%	0.00039148	-0.00000799	-1.9994%
A3	Fixed_Prompt	0.00000033	+0.00000003	+10.3064%	0.00042606	+0.00002660	+6.6593%
A4	RAG_top10*	0.00000027	-0.00000003	-9.9307%	0.00036659	-0.00003287	-8.2288%
A5	RAG_top20*	0.00000028	-0.00000002	-6.2714%	0.00038162	-0.00001784	-4.4654%
A6	RAG_top100*	0.00000018	-0.00000012	-39.1974%	0.00030099	-0.00009847	-24.6501%

Ablation Studies - cpu_burst

cpu_burst

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00097658	—	—	0.01477793	—	—
A1	RAG_PaP	0.00097822	+0.00000163	+0.1672%	0.01476505	-0.00001288	-0.0871%
A2	Random_RAG	0.00097569	-0.00000089	-0.0913%	0.01478225	+0.00000433	+0.0293%
A3	Fixed_Prompt	0.00097518	-0.00000141	-0.1440%	0.01473425	-0.00004367	-0.2955%
A4	RAG_top10*	0.00097658	-0.00000000	-0.0001%	0.01480470	+0.00002677	+0.1812%
A5	RAG_top20*	0.00097731	+0.00000072	+0.0742%	0.01481119	+0.00003326	+0.2251%
A6	RAG_top100*	0.00096573	-0.00001086	-1.1115%	0.01469948	-0.00007844	-0.5308%

Ablation Studies - mem_burst

mem_burst

No	Name	MSE	MSE diff	MSE %	MAE	MAE diff	MAE %
A0	Baseline	0.00000446	—	—	0.00087861	—	—
A1	RAG_PaP	0.00000446	+0.00000001	+0.1449%	0.00086614	-0.00001247	-1.4193%
A2	Random_RAG	0.00000443	-0.00000003	-0.6718%	0.00086920	-0.00000941	-1.0710%
A3	Fixed_Prompt	0.00000447	+0.00000002	+0.3753%	0.00088696	+0.00000835	+0.9501%
A4	RAG_top10*	0.00000443	-0.00000002	-0.5290%	0.00087002	-0.00000860	-0.9783%
A5	RAG_top20*	0.00000445	-0.00000000	-0.0784%	0.00088655	+0.00000793	+0.9029%
A6	RAG_top100*	0.00000437	-0.00000009	-1.9983%	0.00084068	-0.00003794	-4.3178%

분석

RAG_PaP는 일부 핵심 feature에서 Baseline 대비 소폭 개선

- max_mem_usage MAE
  - ***-0.61%***
- mem_burst MAE
  - ***-1.42%***
- cpu_usage MAE
  - ***-0.39%***
- mem_usage MAE
  - ***-0.67%***

Random_RAG와의 차이는 아직 불명확

- 강력한 검색 품질 효과 입증 필요

Fixed_Prompt는 성능 저하 경향

- 의미 없는 고정 텍스트는 예측에 방해 가능
- 단순 고정 프롬프트 추가만으로는 성능 개선이 확인되지 않음

Top-k에 따라 feature별 결과 차이

- cache/burst 계열은 큰 top-k에서 개선
- max_mem_usage는 큰 top-k에서 악화

결론

기여점

a. Time-LLM 기반 클라우드 자원 예측 파이프라인 설계

- Google Cluster Trace 기반 데이터셋 추출 및 전처리
- 클라우드 자원 예측을 위한 도메인 피처 구성

b. RAG를 통한 프롬프트 강화 구조 설계

- FAISS 기반 유사 시계열 검색 DB 구축
- 검색 결과를 Prompt-as-Prefix에 동적으로 주입

한계

a. 정상/안정 구간 중심 데이터셋

- 대부분 안정된 흐름을 보이는 정상 구간 중심 데이터셋
- 급격한 부하 변화 구간에서의 성능 차이 측정 필요

b. 학습 자원 제약으로 인한 데이터 활용 한계

- 제한된 GPU 자원과 학습 시간으로 인해 전체 원천 데이터셋 확장 실험 및 충분한 파라미터 최적화 한계

향후 연구 방향

a. 다양한 데이터셋에 대한 일반화 검증

- 동일 데이터셋 내에서도 다른 Cell 데이터 활용
- Azure, Alibaba 등 다른 Cloud Service Provider

b. RAG DB 구성 전략 고도화

- 시계열 구간의 정규화 방식, 검색 단위, 저장 기준 다양화
- 정교한 classification 기반 RAG DB 구성

c. Retrieved time-series의 직접 활용 방식 검증

- 검색된 유사 시계열을 벡터 형태로 모델 내부에 직접 주입
- LLM 기반 시계열 예측에서 어떤 형태의 외부 기억이 유효한지 검증

d. Backbone 경량화

- SLM 모델로 Backbone 교체 후 학습 시간 및 리소스 효율화

E.O.D